

Date and Time - Simple Hinglish Summary

Source: <https://javascript.info/date>

Main baat

`Date` ek built-in object hai jo **date aur time** store karta hai aur unhe manage karne ke methods deta hai. Iska use creation/modification time store karne, time measure karne, ya current date print karne ke liye hota hai.

1. Date banana (Creation)

`new Date()` ko in arguments ke saath call kar sakte hain:

`new Date()`

Bina arguments ke **abhi ki date aur time**:

javascript

```
let now = new Date();  
alert( now ); // abhi ki date/time
```

`new Date(milliseconds)`

1 Jan 1970 UTC+0 ke baad **milliseconds** (1/1000 second) ginkar date banata hai:

javascript

```
// 0 = 01.01.1970 UTC+0  
let Jan01_1970 = new Date(0);  
alert( Jan01_1970 );  
  
// ab 24 ghante jodo, 02.01.1970 UTC+0 milega  
let Jan02_1970 = new Date(24 * 3600 * 1000);  
alert( Jan02_1970 );
```

1970 ki shuruat se guzre milliseconds ke integer ko **timestamp** kehte hain. Ye date ka halka numeric roop hai. Timestamp se `new Date(timestamp)` se date bana sakte hain, aur Date object se `date.getTime()` se timestamp le sakte hain.

1970 se pehle ki dates ka timestamp **negative** hota hai:

javascript

```
// 31 Dec 1969
let Dec31_1969 = new Date(-24 * 3600 * 1000);
alert( Dec31_1969 );
```

new Date(datestring)

Ek hi argument aur wo string ho to wo **automatically parse** hoti hai (**Date.parse** wala algorithm):

javascript

```
let date = new Date("2017-01-26");
alert(date);
// Time set nahi hai, isliye midnight GMT maana jata hai aur
// jis timezone me code chal raha hai uske hisaab se adjust hota hai
```

new Date(year, month, date, hours, minutes, seconds, ms)

Diye gaye components se **local time zone** me date banata hai. Sirf pehle do arguments zaruri hain.

- **year** 4 digits ka hona chahiye
- **month** 0 se shuru hota hai (0 = Jan, 11 = Dec)
- **date** asal me mahine ka din hai, na de to 1 maana jata hai
- **hours/minutes/seconds/ms** na dein to 0 maane jate hain

javascript

```
new Date(2011, 0, 1, 0, 0, 0, 0); // 1 Jan 2011, 00:00:00
new Date(2011, 0, 1); // wahi, hours etc default 0
```

Maximum precision 1 ms hai:

javascript

```
let date = new Date(2011, 0, 1, 2, 3, 4, 567);
alert( date ); // 1.01.2011, 02:03:04.567
```

2. Date ke components nikalna

- **getFullYear()**: saal (4 digits)
- **getMonth()**: mahina, 0 se 11
- **getDate()**: mahine ka din, 1 se 31 (naam thoda ajeeb hai)
- **getHours()**, **getMinutes()**, **getSeconds()**, **getMilliseconds()**: time ke components

- `getDay()`: hafte ka din, `0` (Sunday) se `6` (Saturday)

`getFullYear()` kabhi mat use karo, wo deprecated hai. Hamesha `getFullYear()`.

Ye saare methods local time zone ke hisaab se dete hain. Inke UTC versions bhi hain (UTC+0 ke liye): `get` ke baad `UTC` lagao, jaise `getUTCFullYear()`, `getUTCMonth()`, `getUTCDay()`.

javascript

```
let date = new Date();

// aapke current time zone ka ghanta
alert( date.getHours() );

// UTC+0 time zone ka ghanta
alert( date.getUTCHours() );
```

Do special methods jinke UTC versions nahi hain:

- `getTime()`: date ka timestamp (1 Jan 1970 UTC+0 se guzre milliseconds)
- `getTimezoneOffset()`: UTC aur local time zone ka farak, **minutes** me

javascript

```
// UTC-1 me ho to 60, UTC+3 me ho to -180
alert( new Date().getTimezoneOffset() );
```

3. Date ke components set karna

- `setFullYear(year, [month], [date])`
- `setMonth(month, [date])`
- `setDate(date)`
- `setHours(hour, [min], [sec], [ms])`
- `setMinutes(min, [sec], [ms])`
- `setSeconds(sec, [ms])`
- `setMilliseconds(ms)`
- `setTime(milliseconds)` (poori date milliseconds se set karta hai)

`setTime()` ko chhodkar sabke UTC variants hain, jaise `setUTCHours()`.

Kuch methods ek saath kai components set kar sakte hain (jaise `setHours()`). Jo components nahi diye wo **badalte nahi**:

javascript

```
let today = new Date();

today.setHours(0);
alert(today); // aaj hi, lekin ghanta 0

today.setHours(0, 0, 0, 0);
alert(today); // aaj, ab theek 00:00:00
```

4. Autocorrection

`Date` objects ki bahut handy khoobi: **out-of-range values dene par wo khud adjust ho jata hai.**

javascript

```
let date = new Date(2013, 0, 32); // 32 Jan 2013 ?!?
alert(date); // ...ye 1 Feb 2013 hai!
```

Maan lo "28 Feb 2016" me 2 din jodne hain. Ye "2 Mar" ya "1 Mar" (leap year ho to) ho sakta hai. Sochne ki zarurat nahi, bas 2 din jodo, `Date` baaki sambhal lega:

javascript

```
let date = new Date(2016, 1, 28);
date.setDate(date.getDate() + 2);

alert( date ); // 1 Mar 2016
```

Ye aksar kisi period ke baad ki date nikalne me use hota hai. Jaise "abhi se 70 seconds baad":

javascript

```
let date = new Date();
date.setSeconds(date.getSeconds() + 70);

alert( date ); // sahi date dikhata hai
```

Zero ya negative values bhi de sakte hain:

javascript

```
let date = new Date(2016, 0, 2); // 2 Jan 2016

date.setDate(1); // mahine ka pehla din
alert( date );

date.setDate(0); // minimum din 1 hai, isliye pichhle mahine ka aakhri din
alert( date ); // 31 Dec 2015
```

5. Date ko number banana, date ka farak

Date ko number me convert karne par wo **timestamp** ban jata hai (`date.getTime()` jaisa):

```
javascript

let date = new Date();
alert(+date); // milliseconds ki sankhya, date.getTime() jaisa
```

Important side effect: dates ko **subtract** kar sakte hain, result unka **milliseconds me farak** hota hai. Isse time measure kar sakte hain:

```
javascript

let start = new Date(); // time naapna shuru

// kaam karo
for (let i = 0; i < 100000; i++) {
  let doSomething = i * i * i;
}

let end = new Date(); // time naapna khatam

alert( `The loop took ${end - start} ms` );
```

6. `Date.now()`

Sirf time measure karna ho to `Date` object banane ki zarurat nahi. `Date.now()` current timestamp return karta hai.

Ye `new Date().getTime()` ke barabar hai, lekin beech ka Date object nahi banata. Isliye **tez hai** aur garbage collection par bojh nahi daalta. Games ya performance-critical jagahon me kaam aata hai.

```
javascript
```

```
let start = Date.now(); // 1 Jan 1970 se milliseconds

// kaam karo
for (let i = 0; i < 1000000; i++) {
  let doSomething = i * i * i;
}

let end = Date.now(); // ho gaya

alert( `The loop took ${end - start} ms` ); // dates nahi, numbers subtract kiy
```

7. Benchmarking

CPU-hungry function ka **bharosemand benchmark** chahiye to dhyan rakhna padta hai.

Maan lo do functions ke speed compare karni hai jo do dates ka farak nikalte hain:

javascript

```
function diffSubtract(date1, date2) {
  return date2 - date1;
}

function diffGetTime(date1, date2) {
  return date2.getTime() - date1.getTime();
}
```

Dono ka result same hai, lekin ek `getTime()` explicitly use karta hai aur dusra date-to-number conversion par bharosa karta hai. Kaunsa tez hai?

Pehla idea: unhe kai baar chalao (kam se kam 100000 baar) aur time naapo:

javascript

```
function bench(f) {
  let date1 = new Date(0);
  let date2 = new Date();

  let start = Date.now();
  for (let i = 0; i < 100000; i++) f(date1, date2);
  return Date.now() - start;
}

alert( 'Time of diffSubtract: ' + bench(diffSubtract) + 'ms' );
alert( 'Time of diffGetTime: ' + bench(diffGetTime) + 'ms' );
```

`getTime()` bahut tez nikla (type conversion nahi hota, engine optimize kar paata hai).
Lekin ye abhi **achha benchmark nahi hai**.

Maan lo `bench(diffSubtract)` chalte waqt CPU kuch aur kar raha tha. Aur `bench(diffGetTime)` ke time wo kaam khatam ho gaya. Modern multi-process OS me ye aam baat hai. Isse pehle benchmark ko kam CPU resources mile, aur galat result aa sakta hai.

Zyada bharsemand benchmarking ke liye poore pack ko kai baar dobara chalana chahiye:

javascript

```
let time1 = 0;
let time2 = 0;

// bench(diffSubtract) aur bench(diffGetTime) ko 10-10 baar baari baari chalao
for (let i = 0; i < 10; i++) {
  time1 += bench(diffSubtract);
  time2 += bench(diffGetTime);
}

alert( 'Total time for diffSubtract: ' + time1 );
alert( 'Total time for diffGetTime: ' + time2 );
```

Modern JS engines advanced optimizations sirf "**hot code**" par lagate hain (jo kai baar chalta hai). Isliye pehle ki runs achhi tarah optimized nahi hoti. Ek "**heat-up**" run add kar sakte hain:

javascript

```
// main loop se pehle "garam" karne ke liye
bench(diffSubtract);
bench(diffGetTime);

// ab benchmark
for (let i = 0; i < 10; i++) {
  time1 += bench(diffSubtract);
  time2 += bench(diffGetTime);
}
```

Microbenchmarking me savdhan raho. Modern engines bahut optimizations karte hain, jo "artificial tests" ke results ko "normal usage" se alag kar sakte hain, khaaskar jab bahut chhoti cheez (jaise ek operator ya built-in function) naapte ho. Performance seriously samajhni ho to pehle JS engine kaise kaam karta hai wo padho, phir shayad microbenchmarks ki zarurat hi na pade.

8. `Date.parse` (string se)

`Date.parse(str)` string se date padhta hai.

String ka format: `YYYY-MM-DDTHH:mm:ss.sssZ`

- `YYYY-MM-DD`: date (saal-mahina-din)
- `"T"`: delimiter
- `HH:mm:ss.sss`: time (ghanta, minute, second, millisecond)
- Optional `'Z'`: time zone, `+ -hh:mm` format me. Akela `Z` = UTC+0

Chhote variants bhi chalte hain: `YYYY-MM-DD`, `YYYY-MM`, ya sirf `YYYY`.

`Date.parse(str)` **timestamp** return karta hai. Format galat ho to `NaN`.

```
javascript

let ms = Date.parse('2012-01-26T13:51:50.417-07:00');

alert(ms); // 1327611110417 (timestamp)
```

Timestamp se turant Date object bana sakte hain:

```
javascript

let date = new Date( Date.parse('2012-01-26T13:51:50.417-07:00') );

alert(date);
```

Summary

- JS me date aur time `Date` object se dikhte hain. "Sirf date" ya "sirf time" nahi bana sakte: `Date` me **hamesha dono** hote hain.
- **Mahine 0 se ginte hain** (January = 0).
- `getDay()` me hafte ke din bhi 0 se (Sunday = 0).
- **Autocorrection:** out-of-range components dene par `Date` khud adjust hota hai. Din/mahine/ghante jodne-ghatane me kaam aata hai.
- Dates ko **subtract** kar sakte ho, result milliseconds me farak. Kyunki number me convert hone par `Date` timestamp ban jata hai.
- Current timestamp tez pane ke liye `Date.now()`.

Dhyan: kai dusre systems ke ulta, JS me timestamps **milliseconds** me hote hain, seconds me nahi.

Kabhi zyada precise time chahiye. JS me `Date` se microseconds nahi milte, lekin zyadatar environments dete hain. Jaise browser me `performance.now()` page load shuru hone se milliseconds deta hai (microsecond precision ke saath):

javascript

```
alert(`Loading started ${performance.now()}ms ago`);  
// jaise: "Loading started 34731.260000000001ms ago"  
// .26 microseconds hain (260 microseconds)  
// decimal ke baad 3 digits se zyada precision errors hain
```

Practice Tasks (Answers)

1. Date banao (20 Feb 2012, 3:12am):

javascript

```
// new Date(year, month, date, hour, minute, second, millisecond)  
let d1 = new Date(2012, 1, 20, 3, 12);  
alert( d1 );  
  
// ya string se  
let d2 = new Date("2012-02-20T03:12");  
alert( d2 );
```

Yaad rakho: February ka number **1** hai (mahine 0 se).

2. Hafte ka din (`getWeekDay()`):

javascript

```
function getWeekDay(date) {
    let days = ['SU', 'MO', 'TU', 'WE', 'TH', 'FR', 'SA'];

    return days[date.getDay()];
}

let date = new Date(2014, 0, 3); // 3 Jan 2014
alert( getWeekDay(date) ); // FR
```

3. European weekday (Monday = 1 se Sunday = 7):

javascript

```
function getLocalDay(date) {

    let day = date.getDay();

    if (day == 0) { // Sunday (0) European me 7 hai
        day = 7;
    }

    return day;
}
```

4. Kitne din pehle mahine ka kaunsa din tha (getDateAgo) (original date nahi badalni chahiye):

javascript

```
function getDateAgo(date, days) {
    let dateCopy = new Date(date);

    dateCopy.setDate(date.getDate() - days);
    return dateCopy.getDate();
}

let date = new Date(2015, 0, 2);

alert( getDateAgo(date, 1) ); // 1, (1 Jan 2015)
alert( getDateAgo(date, 2) ); // 31, (31 Dec 2014)
alert( getDateAgo(date, 365) ); // 2, (2 Jan 2014)
```

Date ki **copy** banana zaruri hai, warna bahar wale code ki date badal jayegi.

5. Mahine ka aakhri din (`getLastDayOfMonth`): Agle mahine ki date banao aur din me 0 do:

javascript

```
function getLastDayOfMonth(year, month) {  
  let date = new Date(year, month + 1, 0);  
  return date.getDate();  
}  
  
alert( getLastDayOfMonth(2012, 0) ); // 31  
alert( getLastDayOfMonth(2012, 1) ); // 29  
alert( getLastDayOfMonth(2013, 1) ); // 28
```

Din `@` ka matlab "1st se ek din pehle", yaani pichhle mahine ka aakhri din.

6. Aaj ke kitne seconds guzre (`getSecondsToday`):

javascript

```
function getSecondsToday() {  
  let now = new Date();  
  
  // aaj ki din/mahina/saal se object banao (00:00:00)  
  let today = new Date(now.getFullYear(), now.getMonth(), now.getDate());  
  
  let diff = now - today; // ms me farak  
  return Math.round(diff / 1000); // seconds  
}
```

Doosra tarika:

javascript

```
function getSecondsToday() {  
  let d = new Date();  
  return d.getHours() * 3600 + d.getMinutes() * 60 + d.getSeconds();  
}
```

7. Kal tak kitne seconds baaki (`getSecondsToTomorrow`):

javascript

```
function getSecondsToTomorrow() {  
  let now = new Date();  
  
  // kal ki date  
  let tomorrow = new Date(now.getFullYear(), now.getMonth(), now.getDate()+1);  
  
  let diff = tomorrow - now; // ms me farak  
  return Math.round(diff / 1000); // seconds me  
}
```

Dhyan: kai deshon me **Daylight Saving Time (DST)** hota hai, isliye 23 ya 25 ghante ke din bhi ho sakte hain.

8. Relative date format karo (`formatDate`):

- 1 second se kam: `"right now"`
- 1 minute se kam: `"n sec. ago"`
- 1 ghante se kam: `"m min. ago"`
- Warna पूरी date `"DD.MM.YY HH:mm"`

javascript

```
function formatDate(date) {
  let diff = new Date() - date; // milliseconds me farak

  if (diff < 1000) { // 1 second se kam
    return 'right now';
  }

  let sec = Math.floor(diff / 1000); // seconds me

  if (sec < 60) {
    return sec + ' sec. ago';
  }

  let min = Math.floor(diff / 60000); // minutes me
  if (min < 60) {
    return min + ' min. ago';
  }

  // date format karo
  // single-digit din/mahina/ghante/minute ke aage 0 lagao
  let d = date;
  d = [
    '0' + d.getDate(),
    '0' + (d.getMonth() + 1),
    '' + d.getFullYear(),
    '0' + d.getHours(),
    '0' + d.getMinutes()
  ].map(component => component.slice(-2)); // har component ke aakhri 2 digits

  // components ko jodo
  return d.slice(0, 3).join('.') + ' ' + d.slice(3).join(':');
}
```

Quick Summary

Kaam	Kaise
Abhi ki date	<code>new Date()</code>
Milliseconds se	<code>new Date(ms)</code>
String se	<code>new Date("2017-01-26")</code>
Components se	<code>new Date(2011, 0, 1)</code> (mahina 0 se)
Saal / mahina / din	<code>getFullYear()</code> , <code>getMonth()</code> , <code>getDate()</code>
Hafta ka din	<code>getDay()</code> (0 = Sunday)
Ghanta / minute / second	<code>getHours()</code> , <code>getMinutes()</code> , <code>getSeconds()</code>
Timestamp	<code>date.getTime()</code> ya <code>+date</code>
Abhi ka timestamp (tez)	<code>Date.now()</code>
Set karna	<code>setFullYear</code> , <code>setMonth</code> , <code>setDate</code> , <code>setHours</code> ...
Din jodna	<code>date.setDate(date.getDate() + 2)</code>
Do dates ka farak	<code>date2 - date1</code> (ms me)
String parse	<code>Date.parse(str)</code> (timestamp / <code>NaN</code>)
UTC version	<code>getUTCHours()</code> , <code>setUTCHours()</code>
Time zone ka farak	<code>getTimezoneOffset()</code> (minutes)

Yaad rakho:

- **Mahina 0 se** aur `getDay()` **0 (Sunday)** se shuru hota hai.
- `getFullYear()` **kabhi nahi**, `getFullYear()` use karo.
- Date badalne se pehle **copy** banao (`new Date(date)`).
- Timestamps JS me **milliseconds** me hote hain.
- Out-of-range values **autocorrect** ho jati hain (mahine ka aakhri din nikalne ka trick: `din[0]`).